

CS Group Project MVC - Code & Presentation Guide

Presentation Reference Document for 5 Modules: Chat, Report, Jasper Report, Notifications, Event Announcements

1. Chat System (Direct & Group Chat with WebSockets)

Overview: Real-time direct & group messaging system using WebSockets for live updates, REST APIs for message operations (send, edit, delete, react, report, block), and database persistence using Hibernate JPA.

File Name	Role / Path	Key Functionality / Notes
ChatController.java	src/main/java/com/hibernate/controller/ChatController.java	REST endpoints for messaging actions (send, edit, delete, react, block, report).
ChatPageController.java	src/main/java/com/hibernate/controller/ChatPageController.java	Page routing for /chat and /chat/room views.
ChatWebSocketHandler.java	src/main/java/com/hibernate/websocket/ChatWebSocketHandler.java	WebSocket connection session handler & text payload routing.
ChatEventBroadcaster.java	src/main/java/com/hibernate/websocket/ChatEventBroadcaster.java	Broadcasts real-time events (new message, typing, reactions, read receipts).
ChatService.java / Impl	src/main/java/com/hibernate/service/ChatServiceImpl.java	Core business logic for conversation creation, message saving & read status.
MessageMapper.java	src/main/java/com/hibernate/service/MessageMapper.java	Converts Message entities into MessageResponse DTOs.
Conversation.java	src/main/java/com/hibernate/entity/Conversation.java	Entity table storing chat room metadata (direct vs group).
ConversationParticipant.java	src/main/java/com/hibernate/entity/ConversationParticipant.java	Mapping table linking users to chat rooms with roles.
Message.java	src/main/java/com/hibernate/entity/Message.java	Entity storing message text, sender, timestamp, and edit flags.
MessageAttachment.java	src/main/java/com/hibernate/entity/MessageAttachment.java	Stores attached file paths and media metadata.
MessageReaction.java	src/main/java/com/hibernate/entity/MessageReaction.java	Stores user emoji reactions on messages.
MessageSeenStatus.java	src/main/java/com/hibernate/entity/MessageSeenStatus.java	Tracks read/seen status for each room participant.
chat.jsp	src/main/webapp/WEB-INF/views/chat/chat.jsp	Main Inbox UI listing active chat conversations.
room.jsp	src/main/webapp/WEB-INF/views/chat/room.jsp	Active Chat Room UI with WebSocket JS client connection.
create-group.jsp	src/main/webapp/WEB-INF/views/chat/create-group.jsp	UI dialog for starting a new group conversation.

Key Functions and Capabilities (What They Can Do)

ChatController.java (REST API Endpoints)

- toggleReaction(messageId, request, session): Toggles a user's emoji reaction on a message (adds if absent, removes if present) and broadcasts the event via WebSockets.
- getInbox(session): Fetches all active direct and group conversations for the logged-in user's inbox list.
- searchUsers(keyword, session): Searches for users by username or display name to initiate new direct chats.
- startDirectChat(userId, session): Initiates a direct message room between the logged-in user and target user (or returns the existing room ID).

- `createConversation(request, session)`: Creates a group conversation with a designated title and user participant IDs.
- `sendMessage(request, session)`: Persists a new text message in the database, handling replies via parent message ID, and triggers WebSocket broadcast.
- `editMessage(messageId, request, session)`: Modifies an existing message's text, tags it as edited, and broadcasts the modification.
- `markMessagesAsRead(request, session)`: Marks all messages in a conversation up to a specific message ID as read for the user, broadcasting read receipts.
- `reportMessage(messageId, request, session)`: Submits a moderation report for a chat message, defining the violation reason and description.
- `deleteConversation(conversationId, session)`: Performs a soft-delete (clears history) of a conversation for the active user.
- `blockUser(userId, session)` / `unblockUser(userId, session)`: Blocks/unblocks a user to restrict or restore direct messaging capabilities.
- `deleteMessage(messageId, session)`: Deletes a message from the database and broadcasts the deletion to room participants.
- `sendMediaMessage(conversationId, caption, files, session)`: Handles multi-file uploads (attachments), saves the media message, and broadcasts it.
- `getChatHistory(conversationId, lastMessageId, session)`: Fetches past messages from a room using pagination for infinite scrolling.
- `getRoomDetails(conversationId, session)`: Retrieves metadata, partner info, and blocking status of a specific chat room.

ChatPageController.java (Page Routing)

- `showChatDashboard(session, model)`: Renders the inbox view showing all active chats.
- `showChatRoom(conversationId, session, model)`: Fetches conversation info, partner statuses, blocking parameters, and renders the chat room page.
- `showCreateGroupForm(session)`: Displays the page to create new group chat conversations.
- `createGroupChat(title, usernames, session, model)`: Validates group title and participant usernames, saves the group chat room, and redirects the user to the new room.

ChatWebSocketHandler.java (WebSocket Routing & Connections)

- `afterConnectionEstablished(session)`: Registers active WebSocket sessions, updates user online status in the database, and broadcasts the status.
- `handleTextMessage(session, message)`: Parses and handles client JSON payloads for typing indicators, stopped typing indicators, message edits, and new message sends.
- `afterConnectionClosed(session, status)`: Unregisters sessions, sets user status to offline if no active connections remain, and broadcasts the status (with last seen time).

ChatEventBroadcaster.java (Event Dispatcher)

- `broadcastPayload(conversationId, payload)`: Dispatches a JSON packet to active WebSocket sessions of conversation participants.
- `broadcastNewMessage(...)` / `broadcastEditedMessage(...)` / `broadcastDeletedMessage(...)`: Transmits message changes to subscribers in real-time.
- `broadcastReaction(...)` / `broadcastReadReceipt(...)`: Delivers reaction changes and read indicators instantly.
- `broadcastUserTyping(...)` / `broadcastUserStoppedTyping(...)`: Delivers active typing states to room participants.
- `broadcastUserStatus(userId, isOnline, lastSeen)`: Broadcasts a user's status change (online/offline) to all active WebSocket sessions globally.

2. Report System (Post, Comment & Message Moderation)

Overview: Multi-tiered reporting system allowing users to report inappropriate posts, comments, or chat messages. Admin dashboard groups reports per item and allows moderation actions (dismiss or delete content).

File Name	Role / Path	Key Functionality / Notes
ReportController.java	src/main/java/com/hibernate/controller/ReportController.java	User API endpoint to submit post reports (/api/reports).
CommentReportController.java	src/main/java/com/hibernate/controller/CommentReportController.java	User API endpoint to report comments (/api/comments/{id}/report).
AdminReportController.java	src/main/java/com/hibernate/controller/AdminReportController.java	Admin Controller to review grouped reports & execute dismiss/delete actions.
ReportService / Impl	src/main/java/com/hibernate/service/ReportServiceImpl.java	Groups multiple reports by target post, calculates counts and reason lists.
AdminCommentService / Impl	src/main/java/com/hibernate/service/AdminCommentServiceImpl.java	Service logic for fetching and managing grouped comment reports.
PostReport.java	src/main/java/com/hibernate/entity/PostReport.java	Entity table storing post report entries.
CommentReport.java	src/main/java/com/hibernate/entity/CommentReport.java	Entity table storing comment report entries.
GroupedPostReportDto.java	src/main/java/com/hibernate/dto/GroupedPostReportDto.java	DTO containing post info, total report count, and list of reporter reasons.
GroupedCommentReportDto.java	src/main/java/com/hibernate/dto/GroupedCommentReportDto.java	DTO containing comment info and aggregated report data.
reports.jsp	src/main/webapp/WEB-INF/views/admin/reports.jsp	Admin Moderation Dashboard UI for reviewing reported posts and comments.

Key Functions and Capabilities (What They Can Do)

ReportController.java (Post Reporting)

- reportPost(id, reason, description, session): Saves a post report record, saving the reporting user ID, target post ID, violation category, and optional custom text.

CommentReportController.java (Comment Reporting)

- reportComment(id, reason, description, session): Saves a comment report record for a comment ID, logging the violator and description.

AdminReportController.java (Moderation Dashboard & Actions)

- viewReports(type, view, model, session): Renders the reports list filtered by type (posts/comments) and view (active queue/historical reports).
- dismissPostReport(id, session) / dismissCommentReport(id, session): Dismisses a specific report by marking the item as safe in the database.
- resolvePostReport(id, reason, session) / resolveCommentReport(id, reason, session): Resolves a single report by marking it resolved and deleting the reported content.
- dismissGroupedPostReport(postId, session) / dismissGroupedCommentReport(commentId, session): Dismisses all active reports targeting a specific post or comment in bulk.
- resolveGroupedPostReport(postId, reason, duration, banType, session) / resolveGroupedCommentReport(commentId, reason, duration, banType, session): Bulk-resolves reports by deleting the post or comment and applying a temporary user suspension (banning the user's posting, commenting, or all privileges) for a set duration.

ReportService.java (Moderation Business Logic)

- getGroupedPendingPostReports() / getGroupedPendingCommentReports(): Groups database reports by target content and wraps them inside DTOs, counting total reports and aggregating reporter reasons for cleaner UI listing.
- dismissAllPostReportsByPostId(adminId, postId) / dismissAllCommentReportsByCommentId(...): Performs database updates to change status of all reports on a specific content to DISMISSED.
- resolveAllPostReportsByPostId(...) / resolveAllCommentReportsByCommentId(...): Marks reports as RESOLVED, soft-deletes the target item, and inserts a user suspension row.

3. Jasper Report Generation (Monthly CheatSheet Report)

Overview: Analytics PDF report generation using JasperReports library. Fetches monthly data, compiles XML template (.jrxml), populates DTO fields, and streams PDF directly to admin browser.

File Name	Role / Path	Key Functionality / Notes
CheatSheetReportController.java	src/main/java/com/hibernate/controller/CheatSheetReportController.java	Admin REST endpoints (/admin/reports/cheatsheet/pdf) to trigger PDF export.
CheatSheetReportService / Impl	src/main/java/com/hibernate/service/CheatSheetReportServiceImpl.java	Compiles JRXML template, fills data source, and converts report to byte[] PDF.
CheatSheetReportDto.java	src/main/java/com/hibernate/dto/CheatSheetReportDto.java	Data Transfer Object holding total posts, total views, active users, top posts.
monthly_cheat_sheet_report.jrxml	src/main/resources/reports/monthly_cheat_sheet_report.jrxml	Jasper Studio XML design template defining tables, headers, styling.
cheatsheet-report.jsp	src/main/webapp/WEB-INF/views/admin/cheatsheet-report.jsp	Admin page UI to select month/year and preview/download PDF reports.

Key Functions and Capabilities (What They Can Do)

CheatSheetReportController.java (Report Request Controller)

- viewCheatSheetReport(page, pageSize, model, session): Fetches paginated cheat sheet statistics list and renders the dashboard UI.
- downloadPdf(session): Invokes PDF generation and returns the binary array as an inline PDF attachment stream to the browser.
- downloadExcel(session): Invokes Excel generation and returns the binary array as a downloadable spreadsheet.

CheatSheetReportService.java (Jasper Report Generator)

- getReportData(): Queries the database for monthly metrics, including total cheat sheets, active writers, cumulative view counts, and active post trends.
- getTotalCheatSheetsCount(): Computes the total quantity of cheat sheets saved in the database.
- generatePdfReport(): Loads the Jasper template (.jrxml), fills it with the compiled data DTOs using JasperFillManager, processes it into PDF format, and returns the byte array.
- generateExcelReport(): Exports the compiled report data as an Excel binary stream.

4. Notification System (User Notifications & Advice)

Overview: In-app notification mechanism triggering alerts for likes, comments, approvals, and announcements. Uses @ControllerAdvice to inject unread counts into all view headers globally.

File Name	Role / Path	Key Functionality / Notes
NotificationController.java	src/main/java/com/hibernate/controller/NotificationController.java	REST endpoints (/api/notifications) for fetching user notifications & mark-read.
NotificationAdvice.java	src/main/java/com/hibernate/controller/NotificationAdvice.java	@ControllerAdvice providing global unread notification count to all JSP views.
NotificationService / Impl	src/main/java/com/hibernate/service/NotificationServiceImpl.java	Core service for building, linking target URLs, and persisting notifications.
Notification.java	src/main/java/com/hibernate/entity/Notification.java	Entity table storing notification message, recipient, sender, type, and read flag.
NotificationRepository.java	src/main/java/com/hibernate/repository/NotificationRepository.java	JPA Repository fetching user notifications sorted by creation date.
notifications.jsp	src/main/webapp/WEB-INF/views/notifications/notifications.jsp	User notification list screen.

Key Functions and Capabilities (What They Can Do)

NotificationController.java (User Notifications Endpoints)

- listNotifications(session, model): Fetches all notifications for the active user and renders the notifications list page.
- markAsRead(id, session): Marks a specific notification as read and redirects back to notifications.
- markAllAsRead(session): Marks all unread notifications of the user as read.

NotificationAdvice.java (Global Controller Interceptor)

- addUnreadNotificationCount(session, model): Runs globally before all controllers. It checks if the current user has been fully banned, automatically logging them out if so. If not, it updates the session and model attributes with the fresh database state (dbUser) and binds the global unreadNotificationCount and totalUnreadChatCount badges to display in the header menu.

NotificationService.java (Notification Builder & Repository Manager)

- createNotification(userId, type, title, message, referenceType, referenceId): Constructs a new notification instance (e.g. LIKE, COMMENT, APPROVE) with target URLs, saving it to the database.
- broadcastNotification(type, title, message, referenceType, referenceId): Instantiates and saves a notification to all registered active users in the system.
- getNotificationsForUser(userId): Queries all user notification records, sorted by date (newest first).
- getUnreadCount(userId): Returns the number of unread notifications for a user.
- markAsRead(notificationId, userId) / markAllAsRead(userId): Updates read status flags to true in the database.

5. Event Announcement System

Overview: System-wide announcement management by Admins. Creating an announcement automatically dispatches notifications to all target users and displays banners on user feeds.

File Name	Role / Path	Key Functionality / Notes
AdminAnnouncementController.java	src/main/java/com/hibernate/controller/AdminAnnouncementController.java	Admin REST endpoints (/admin/announcements) for CRUD operations.
AnnouncementService / Impl	src/main/java/com/hibernate/service/AnnouncementServiceImpl.java	Saves announcement and calls NotificationService to alert all users.
Announcement.java	src/main/java/com/hibernate/entity/Announcement.java	Entity table storing title, content, target role, publication status, timestamps.
AnnouncementRepository.java	src/main/java/com/hibernate/repository/AnnouncementRepository.java	JPA Repository for querying active published announcements.
announcements.jsp	src/main/webapp/WEB-INF/views/admin/announcements.jsp	Admin UI for drafting, publishing, and managing announcements.

Key Functions and Capabilities (What They Can Do)

AdminAnnouncementController.java (Admin Announcement CRUD)

- listAnnouncements(...) / createAnnouncement(...) / deleteAnnouncement(...): CRUD endpoints allowing administrators to draft, edit, publish, or remove global announcements.

AnnouncementService.java (Announcement Broadcast Coordinator)

- publishAnnouncement(announcement): Saves the announcement and calls NotificationService.broadcastNotification(...) to alert all target users.

Presentation & Teacher Defense Key Points

1. Architecture Pattern: The application follows the standard Spring MVC Architecture (Controller Service Repository Database Entity View/JSP). REST endpoints return JSON/PDF streams while page controllers render JSP views.
2. Real-time Communication (WebSocket): Chat functionality uses ChatWebSocketHandler for bidirectional low-latency messaging, combined with ChatService for DB persistence.
3. Grouped Moderation Logic: To prevent duplicate UI items in admin moderation, GroupedPostReportDto aggregates multiple reports targeting the same post into a single row with total report counts and concatenated reasons.
4. Jasper Export Flow: Monthly metrics are compiled into a CheatSheetReportDto, passed to JasperFillManager alongside monthly_cheat_sheet_report.jrxml, and rendered as a PDF byte stream via HTTP Response.
5. Global Notification Advice: NotificationAdvice uses Spring's @ControllerAdvice to inject the unread notification count model attribute into every JSP automatically without repeating code in each controller.